Graphics & Games Cheat Sheet

GL 4.6 · WebGL 2 · Metal 4 · Vulkan 1.4 · Unity 6.3 LTS · UE 5.8 · checked 5 Nov 2026 · Irurueta Docs

The six libraries

Library	Version	Language	Licence
OpenGL	4.6, ES 3.2	C, C++	royalty-free spec
WebGL	2.0, three.js r187	JS, TS	royalty-free
Metal	4, MSL 4.1	Swift, ObjC, C++	free, proprietary
Vulkan	1.4	C, C++	royalty-free
Unity	6.3 LTS	C#	free tier, paid seats
Unreal	5.8, 5.8.4	C++, Blueprints	5% above US$1M

WebGPU: W3C Candidate Recommendation Draft.

Pipeline stages

input → vertex → tess → geom → clip → raster → fragment → depth, stencil → blend

API	Programmable stage names
OpenGL, Vulkan	vertex, tess control, tess evaluation, geometry, fragment; task and mesh by extension
Metal	vertex function, post-tessellation vertex, fragment function; object and mesh; no geometry
WebGPU, WebGL	vertex, fragment; WebGPU adds compute
HLSL	vertex, hull, domain, geometry, pixel

Coordinate chain

object M world V view P clip /w NDC vp window

API	Clip Z	NDC Y	View
OpenGL, WebGL	−1..1	up	RH, −Z
Metal, WebGPU	0..1	up	library choice
Vulkan	0..1	down	library choice
Direct3D	0..1	up	LH, +Z
glTF	RH, +Y up, front +Z, metres		

MVP and perspective

clip = P V M p (column vectors; M applies first). M = T R S.

c = cot(fov/2), a = width/height, n near, f far:

```
| c/a 0   0       0      |
|  0  c   0       0      |
|  0  0   A       B      |
|  0  0  −1       0      |
GL  −1..1: A=(f+n)/(n−f) B=2fn/(n−f)
0..1:      A=f/(n−f)     B=nf/(n−f)
```

Normals: inverse transpose of M. Reversed-Z gives better depth precision.

Quaternions

q = (u sin(θ/2), cos(θ/2)), u a unit axis; v′ = q v q*.

slerp(a,b,t) = [sin((1−t)Ω) a + sin(tΩ) b] / sin Ω, cos Ω = a·b.

If a·b < 0 negate b; if near 1 use normalised lerp. Euler angles: gimbal lock.

Lighting

Blinn-Phong, H = norm(L+V):

$I = k_a + k_d \max(N\cdot L,0) + k_s \max(N\cdot H,0)^s$

glTF PBR, α = roughness^2:

- $F = F_0 + (1-F_0)(1-V\cdot H)^5$; F_0 = mix(0.04, base, metallic)
- $D = \alpha^2 / (\pi((N\cdot H)^2(\alpha^2-1)+1)^2)$ (GGX)
- Vis = Smith G / (4 |N·L| |N·V|)
- f = F D Vis + (1−F) c_{diff}/π; c_{diff} = mix(base, 0, metallic)

Shadows: depth map, bias against acne, PCF, cascades.

sRGB and HDR

Decode: c ≤ 0.04045 ? c/12.92 : $((c+0.055)/1.055)^{2.4}$

Encode: c ≤ 0.0031308 ? 12.92c : $1.055c^{1/2.4} - 0.055$

Light in linear, tone map (Reinhard, ACES, Khronos PBR Neutral), then encode. Colour textures sRGB; normal, metal and roughness linear.

Textures

Filters: nearest, bilinear, trilinear (mips), anisotropic.

Mips: $\lfloor \log_2 \max(w,h) \rfloor + 1$ levels; about 33% more memory.

Desktop, consoles	BC1 to BC7
Android, Arm	ETC2, ASTC
Apple	ASTC; BC on Macs
Web, any	KTX 2.0 with Basis Universal, transcoded at load

Shading languages

Source	IR	API
GLSL, ESSL	none, or SPIR-V	OpenGL, ES, WebGL, Vulkan
HLSL	DXIL, SPIR-V	Direct3D, Vulkan; Unity, Unreal
MSL	metallib	Metal
WGSL	none	WebGPU
Slang	SPIR-V, DXIL, others	Vulkan, Direct3D, Metal

Cross-compile: SPIRV-Cross, Naga, Tint.

Asset formats

glTF 2.0	runtime delivery, PBR, open, ISO/IEC 12113:2022; GLB is the binary form
OpenUSD	composition of large scenes; USDZ on Apple; open
FBX	proprietary, common in engine pipelines
OBJ, MTL	legacy text; geometry only, no animation

Fixed-timestep loop

```
acc = 0; prev = now()
loop:
  t = now(); acc += min(t−prev, maxStep)
  prev = t; input()
  while acc >= dt:
    save(); update(dt); acc −= dt
  render(acc / dt)   # interpolate
```

ECS: entities are IDs, components data, systems logic.

OpenXR object chain

instance → system → session → space → swapchain

Loader finds the runtime. Each frame: wait, begin, render one view per eye, end.

Translation layers

ANGLE	OpenGL ES on Vulkan, D3D, Metal
MoltenVK, KosmicKrisp	Vulkan on Metal
Dawn, wgpu	WebGPU on native APIs
DXVK, vkd3d-proton	Direct3D on Vulkan

Profilers

Vulkan, D3D, GL, ES	RenderDoc (no Metal)
Apple	Xcode Metal debugger, Instruments
Windows	PIX
Android	Android GPU Inspector
Browser	DevTools, Spector.js
Engines	Unity Profiler, Unreal Insights

CPU- or GPU-bound first; then draw calls, overdraw, bandwidth.

Choosing

Ship a game	**Unity** or **Unreal**
Browser	**three.js**, WebGL, WebGPU
Apple only	**Metal**, RealityKit
Maximum control	**Vulkan**
Learning	**OpenGL**
XR	an engine with OpenXR

GPU and API model

Immediate-mode	draws triangles as they arrive; desktop GPUs
Tile-based	bins into tiles, then shades; mobile and Apple; load and store actions cost bandwidth
Implicit	OpenGL state machine; the driver tracks memory and sync
Explicit	Vulkan, Metal, WebGPU: pipeline objects, bind groups, command buffers; Vulkan also leaves memory and sync to you

Meshes

Vertices plus indices; a hard edge needs split vertices, so a box has 24 vertices and 36 indices. Winding decides the front face and culling. Attributes: position, normal, tangent, UV. Instancing draws one mesh many times; LOD swaps by distance.

Blending and post-processing

Over: out = src.a · src + (1−src.a) · dst.

Premultiplied: out = src + (1−src.a) · dst.

Sort transparent objects back to front, or use OIT. Antialiasing: MSAA, FXAA, TAA. Forward, Forward+ or deferred with a G-buffer. Post chain: bloom, tone map, encode.

Physics, audio, input

Broad phase (bounds) then narrow phase (shapes), then integrate. Audio: a listener and positioned sources. Input: map actions, not keys.

Advanced stages

Compute: general GPU work. Mesh and task shaders replace input and vertex stages. Ray tracing: a BVH with ray-gen, hit and miss shaders, or ray queries. Avoid geometry shaders.